

Fixed odds API overview

When the GameWrapper is initialized, you can access the RGS features through the `GameWrapper.prototype.rgs` member.

The interface and the available methods depend on the `gameType` set during initialization. This document describes the methods and their responses for *'fixed-odds'* games.

Note: To avoid blocked requests, it is recommended to check `GameWrapper.prototype.state` before calling RGS methods and only call them if the state is `active`. If the state is `paused`, the client should wait for a `game-resume` event before calling the desired method.

Common interfaces

PrizeLevel

```
{
  level: number,
  probabilityUp: number,
  probabilityDown: number,
  winAmount: number,
  freeTicket: boolean,
  scenarios: number
}
```

RgsGameData

```
{
  gameId: string,
  isSuspended: boolean,
  gameEngine: "fixedOdds",
  gameEngineConfiguration: null,
  ticketPrice: number[],
  allowedNumberOfTickets: number[],
  prize: PrizeLevel[],
  openTicketLimiterStrategy: string
}
```

RgsAccountData

```
{
  balance: number,
  currency: string
}
```

RgsTicketData

```
{
  ticketId: string,
  state: "P" | "S" | "C" | "U" | "V",
  playerId: string,
  gameId: string,
  ticketPrice: number,
  tsPurchased: string,
  tsLastUpdate: string,
  tsSettled: string | null,
  winAmount: number,
  prizes: PrizeLevel[],
  context: {
    playerId: string,
    data: {
      currency?: string,
      currentBalance?: number,
      ipAddr?: string,
      walletMessageAvailable?: string | boolean
    }
  }
}
```

Methods

- `getGame()`
- `getAccount()`
- `getUnsettledTicket()`
- `purchaseTicket(bet: number)`
- `getTicketData(ticketId?: string)`
- `setTicketData(data: string | object, ticketId?: string)`
- `settleTicket()`
- `getTicket(ticketId: string)`
- `sendCustomRequest(service: string, method: string, args: any[])`

`getGame()`

Get game data including stake, prize level and configuration data. `getGame()` should be the first RGS API call after `GameWrapper` is initialized.

Return value

- `Promise<RgsGameData>` - a promise that resolves to `RgsGameData` once a valid response is received. In case of failure, the promise will be rejected

with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getGame()
  .then(function (data) {
    console.log('Available stakes: ' + data.ticketPrice);
  }).catch(handleErrors);
```

Example response

```
{
  gameId: "25-card-cash",
  isSuspended: false,
  gameEngine: "fixedOdds",
  gameEngineConfiguration: null,
  ticketPrice: [0.2, 0.4, 1, 2, 3, 5, 10, 15],
  allowedNumberOfTickets: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
  prize: [
    {
      level: 0,
      probabilityUp: 72520989,
      probabilityDown: 100000000,
      winAmount: 0,
      freeTicket: false,
      scenarios: 610
    },
    ...
  ],
  openTicketLimiterStrategy: "{playerId}:{gameId}",
  slotPayTable: {
    winPayTable: null,
    scatterPayTable: null
  }
}
```

getAccount()

Get user's current balance data including the currency. Calling this method will trigger a balance update throughout the whole GameWrapper.

Return value

- **Promise<RgsAccountData>** - a promise that resolves to **RgsAccountData** once a valid response is received. In case of failure, the promise will be

rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getAccount()
  .then(function (data) {
    console.log('Current balance: ' + data.balance + ' ' + data.currency);
  }).catch(handleErrors);
```

Example response

```
{
  balance: 10000,
  currency: "GBP"
}
```

getUnsettledTicket()

Get an unsettled ticket if it exists. An unsettled ticket may exist if the previous game hasn't been settled properly (e.g. if the user closed the game early, or the game crashed).

It is recommended to call this method before calling `purchaseTicket()`, but it is not required. The wrapper will execute `getUnsettledTicket()` automatically if purchase fails due to an unsettled ticket.

Return value

- `Promise<RgsTicketData>` - a promise that resolves to `RgsTicketData` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getUnsettledTicket()
  .then(function (ticket) {
    if(ticket)
      console.log('Unsettled ticket found. Details: ' + JSON.stringify(ticket))
  }).catch(handleErrors);
```

Example Response

```
{
  ticketId: "ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0",
  state: "P",
  playerId: "1550069350",
}
```

```

gameId: "25-card-cash",
ticketPrice: 0.2,
tsPurchased: "2019-02-13T15:57:48.734943998+01:00",
tsSettled: null,
winAmount: 0,
prizes: [{
  level: 0,
  winAmount: 0,
  scenario: "scenario"
}],
context: {
  playerId: "demo:1550069350",
  data: {
    currency: "GBP",
    currentBalance: 10000,
    ipAddr: "127.0.0.1",
    walletMessageAvailable: "false"
  }
},
tsLastUpdate: "2019-02-13T15:57:48.734944217+01:00"
}

```

purchaseTicket(bet: number)

Purchase a new ticket. In case of a successful purchase, the response will contain ticket data including the selected scenario.

If there is an unsettled ticket, no purchase will be made, but the existing ticket will be returned as if `getUnsettledTicket()` was called.

Argument

- `bet` - {number} ticket price

Return value

- `Promise<RgsTicketData>` - a promise that resolves to `RgsTicketData` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see [Error Handling Guidelines](#).

Usage example

```

wrapper.rgs.purchaseTicket(0.5)
  .then(function (ticket) {
    console.log('Selected scenario: ' + ticket.scenario);
  }).catch(handleErrors);

```

Example response

```
{
  ticketId: "993ce890-83b0-11e9-9560-000c2932a097",
  state: "P",
  playerId: "5ceec2e98052640001f880ff",
  gameId: "mayan-farmer",
  ticketPrice: 0.5,
  tsPurchased: "2019-05-31T14:30:11.989933057Z",
  tsSettled: null,
  winAmount: 1,
  scenario: "{\"r\": \"c,p,p,p,p,c,t,c,c,p,c,p,t,t,p\", \"w\": \"4p=2,3c=4,3p=8\"}",
  prize: {
    level: 9,
    winAmount: 1,
    freeTicket: false,
    scenarios: 0
  },
  prizes: [
    {
      level: 9,
      winAmount: 1,
      scenario: "{\"r\": \"c,p,p,p,p,c,t,c,c,p,c,p,t,t,p\", \"w\": \"4p=2,3c=4,3p=8\"}"
    }
  ],
  context: {
    playerId: "5ceec2e98052640001f880ff",
    data: {
      currency: "GBP",
      ipAddr: "127.0.0.1",
      machineId: "mayan-farmer",
      sessionToken: "jFONk8dKfIfQf90PR0xHtkwX42Ax16_6BefysaP90LE="
    }
  },
  tsLastUpdate: "2019-05-31T14:30:11.989966883Z"
}
```

getTicketData(ticketId?: string)

Get data that was associated with a certain ticket.

If `ticketId` isn't specified, the wrapper will get data associated with the current ticket.

Argument

- `ticketId` - {string} ticket ID for which to get data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was previously set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicketData('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) {
    console.log('Ticket data: ' + data);
  }).catch(handleErrors);
```

Example response

```
'{"color": "red"}'
```

`setTicketData(data: string | object, ticketId?: string)`

Associate data with a ticket.

If `ticketId` isn't set it will default to the current ticket ID.

Arguments

- `data` - `{string | object}` the data to be associated with a ticket. May either be a JSON string or a serializable javascript object. The object will be stringified before sending the request to RGS.
- `ticketId` - `{string}` ticket ID for which to set data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.setTicketData({ color: 'red' })
  .then(function (data) {
    console.log('Ticket data: ' + data);
  }).catch(handleErrors);
```

Example response

```
'{"color": "red"}'
```

settleTicket()

Settle the current ticket. **settleTicket** should be called at the end of each game, before another purchase is made.

Return value

- **Promise<RgsTicketData>** - a promise that resolves to **RgsTicketData** once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see [Error Handling Guidelines](#).

Usage example

```
wrapper.rgs.settleTicket('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) {
    console.log('Ticket settled');
  }).catch(handleErrors)
```

Example response

```
{
  ticketId: "ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0",
  state: "S",
  playerId: "1550069350",
  gameId: "25-card-cash",
  ticketPrice: 0.2,
  tsPurchased: "2019-02-13T15:57:48.734943998+01:00",
  tsSettled: "2019-02-13T15:58:48.734943998+01:00",
  winAmount: 0,
  prizes: [{
    level: 0,
    winAmount: 0,
    scenario: "scenario"
  }],
  context: {
    playerId: "demo:1550069350",
    data: {
      currency: "GBP",
      currentBalance: 10000,
      ipAddr: "127.0.0.1",
      walletMessageAvailable: "false"
    }
  },
  tsLastUpdate: "2019-02-13T15:58:48.734944217+01:00"
}
```


getTicket(ticketId: string)

Get ticket data for a specific settled ticket. This is useful for getting historical ticket data, though this is usually handled by the GameWrapper and there is no for the client to call this method. For more details about history replay, see the appropriate section below.

Argument

- ticketId - {string} - ticket id of the ticket to settle

Return value

- Promise<RgsTicketData> - a promise that resolves to RgsTicketData once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicket('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) { console.log('Ticket has scenario: '+ticket.scenario)})
  .catch(handleErrors)
```

Example response

```
{
  ticketId: "ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0",
  state: "S",
  playerId: "1550069350",
  gameId: "25-card-cash",
  ticketPrice: 0.2,
  tsPurchased: "2019-02-13T15:57:48.734943998+01:00",
  tsSettled: "2019-02-13T15:58:48.734943998+01:00",
  winAmount: 0,
  prizes: [{
    level: 0,
    winAmount: 0,
    scenario: "scenario"
  }],
  context: {
    playerId: "demo:1550069350",
    data: {
      currency: "GBP",
      currentBalance: 10000,
      ipAddr: "127.0.0.1",
      walletMessageAvailable: "false"
    }
  }
},
```

```

    tsLastUpdate: "2019-02-13T15:58:48.734944217+01:00"
  }

```

sendCustomRequest(service: string, method: string, args: any[])

Send a custom RGS request through the wrapper. This is useful when the client wants to use a feature which is supported by the RGS, but not implemented by the wrapper.

Should be used with caution. Best course of action is to ask first if this is the right way to implement required functionality. That is to avoid unnecessary debugging and delays.

Argument

- **service** - {string} RGS service name
- **method** - {string} RGS method name
- **args** - {any[]} array of arguments for the method. May be an empty array for no arguments.

Return value

- **Promise<any>** - a promise that resolves to the response object once a valid response is received. The exact response structure depends on the RGS method that is being called. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```

const gameId = /* some gameId related logic */;
wrapper.rgs.sendCustomRequest('SlotService', 'Refresh', [gameId])
  .then(function (data) { console.log('Refresh response: ', data) })
  .catch(handleErrors);

```